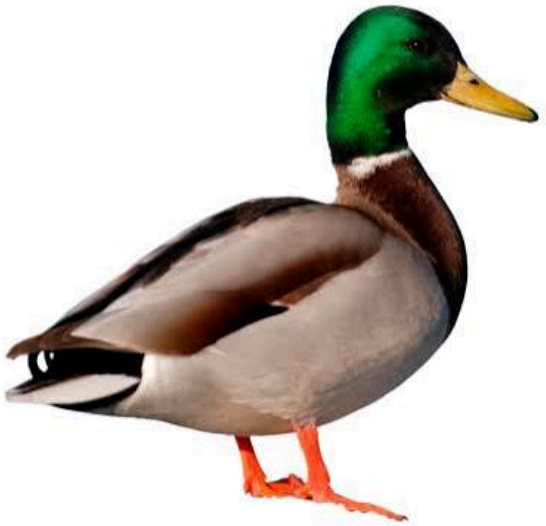
Ciência de Dados Aplicada ao Direito I

Aula 22

Do canabidiol ao Tema 1.417



Prof. Bruno da Cunha de Oliveira - 2026/2

Onde estamos

Insper

O que já respondemos à empresa de canabidiol?

A empresa fez três perguntas sobre as 300 sentenças do TJSP: contra quem se judicializa mais, quem é mais acionado em cada grupo, e quem ganha.

Para cada uma: como respondemos, e o que ficou faltando?

Três perguntas, três regras, três ressalvas

RESPOSTA

Pergunta	Como respondemos	O que faltou
Público ou privado?	regex do réu e regra por palavras: 196 de 300 contra o Estado	12 sem réu; "canabidiol" puxa o SUS
Quem é mais acionado?	contar o réu	a mesma Fazenda escrita de três jeitos
Quem ganha?	regex do dispositivo	40 de 300 sem classe

Outra maneira: utilizando Inteligência Artificial

Quem mais poderia preencher as três colunas?

Em vez de descrever o padrão do texto, dá para pedir a alguém que leia cada sentença e responda: quem é o réu, se é público ou privado, qual foi o resultado.

Ler 300 à mão leva um dia. Quem mais lê texto e responde em formato fixo?

Um modelo de linguagem

RESPOSTA

Um modelo de linguagem lê a sentença e preenche o formulário. Chamado de dentro do programa, ele faz isso para cada linha da tabela.

Para isso precisamos de três coisas: uma chave de acesso a um serviço que roda o modelo, um formulário com as perguntas, e a biblioteca que liga os dois à tabela.

Como o modelo escreve

O texto vira tokens

Animação no navegador: `animacao-tokens.html`, cenário `quebrar`.

A próxima palavra de uma ementa

```
EMENTA: Apelação cível. Plano de saúde.
Negativa de cobertura. Recurso ...
```

Que palavra vem depois de "Recurso"? Deem também uma chance, em porcentagem, para o seu palpite.

Um token de cada vez

Animação no navegador: `animacao-tokens.html`, cenário `gerar`.

Um preditor do próximo token

Dado o texto até aqui, o modelo dá uma chance a cada token possível e escolhe o próximo. Depois repete.

O treino fica cada vez mais sofisticado, mas o que o modelo faz ao responder é isso. Preencher um formulário é escrever, token a token, um texto no formato que o formulário pede.

Criar a conta e a chave na Groq

#	O que fazer
1	Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail
2	No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1`
3	Copiar a chave agora: ela começa com `gsk_`

A chave diz à Groq quem está pedindo: pedido, cota e cobrança caem na conta de quem a criou. Não mandem a ninguém, nem para mim.

Onde guardar a chave

O jeito mais rápido é colar a chave no próprio programa:

```
chave = "gsk_..."
```

O programa funciona. O que pode dar errado?

A chave escrita no código

Animação no navegador: `animacao-api-llm.html`, cenário `codigo`.

A chave vai junto com o arquivo

RESPOSTA

O arquivo vai para o repositório, e a chave vai junto: no repositório da fase 2, quem o abrir usa a conta de vocês. Apagar a linha e enviar de novo não resolve, porque a versão anterior continua no histórico.

Chave que vazou se apaga no painel da Groq, em **API Keys**, e se troca por outra.

A chave no arquivo .env

#	O que fazer, na pasta `cdad1`
1	No terminal do VS Code: `uv add python-dotenv`
2	Criar o arquivo `.env`, com uma linha só: `GROQ_API_KEY=` e a sua chave colada depois do =
3	Abrir o arquivo `.gitignore` e acrescentar uma linha com `.env`

O `.gitignore` lista o que o Git não registra: o `.env` fica no computador e nunca vai para o GitHub.

Conferir a chave

```python
import os
from dotenv import load_dotenv

load_dotenv()
chave = os.environ["GROQ_API_KEY"]
print(chave[:4])
```

No caderno da aula 22, passo 1. O que aparece? E se o `.env` não existisse?

os.environ lê o ambiente

RESPOSTA

```
gsk_
```

O `load_dotenv()` leva o que está no `.env` para o ambiente, e `os.environ` é um dicionário com ele. Sem o arquivo, a chave não existe:

```
KeyError: 'GROQ_API_KEY'
```

Só os quatro primeiros caracteres vão para a tela: a chave inteira nunca aparece num print.

O modelo preenche a tabela

Ler o README de uma biblioteca

A biblioteca é o dataframeit, no GitHub. O README de uma biblioteca costuma ter o que basta para a primeira chamada: como instalar, um exemplo mínimo e a lista de argumentos.

```
uv add "dataframeit[groq]"
```

Vamos ler juntos o exemplo do README e adaptá-lo às nossas três perguntas.

Primeiro formulário: tudo em texto

```
class Formulario(BaseModel):
    requerido: str
    polo_passivo: str
    resultado: str
```

Um campo por linha: nome, dois-pontos e tipo; `str` é texto livre. A classe herda de `BaseModel` o trabalho de conferir cada resposta. O pedido: "me diga quem é o requerido, se ele é público ou privado, e qual foi o resultado".

O que volta nas dez?

Com esse formulário, o modelo preenche as três colunas das dez primeiras sentenças.

Dá para contar quantas são procedentes direto na coluna `resultado`?

Dez respostas sem padrão

RESPOSTA

requerido	polo_passivo	resultado
Público	Fazenda Pública... e o Município de Marília	Procedente – condenação solidária...
UNIMED DE GUARULHOS... (entidade privada de saúde)	Requerido	Ação parcialmente procedente: a liminar...

Fechar as opções: `Literal`

```python
from typing import Literal
from pydantic import BaseModel
class Formulario(BaseModel):
    requerido: str
    polo_passivo: Literal["público", "privado"]
    resultado: Literal["procedente",
                       "parcialmente procedente",
                       "improcedente",
                       "extinto sem julgamento do mérito"]
```

O que `Literal` muda

`Literal` fecha a lista de valores que o modelo pode devolver, e a classe recusa o que estiver fora dela. "Público" e "público" deixam de ser duas respostas.

Com a lista fechada, duas rodadas do mesmo modelo, ou dois modelos diferentes, ficam mais próximos: a resposta passa a ser replicável.

Uma chamada para dez sentenças

```
from dataframeit import dataframeit
dez = decisoes.head(10)
tabela = dataframeit(dez, Formulario,
    "Leia a sentença e preencha os campos: {texto}",
    provider="groq", text_column="texto")
print(tabela[["requerido", "polo_passivo", "resultado"]])
```

`head(10)` separa as dez primeiras das 300, para a chamada ser curta. Uma linha de cada vez: o texto vai a um servidor da Groq, o modelo responde, a classe confere. A chave não aparece: o dataframeit a lê do ambiente.

Os argumentos do dataframeit

Argumento	O que é
`dez`	as dez primeiras sentenças, uma por linha
`Formulario`	o formulário, que confere cada resposta
o texto com `{texto}`	o pedido ao modelo; cada sentença entra no lugar de `{texto}`
`provider="groq"`	o serviço que roda o modelo; a chave vem de `GROQ_API_KEY`
`text_column="texto"`	a coluna em que está o texto

Rodem nas dez

No caderno, passos 5 e 6. Na conta gratuita, as dez levam uns três minutos.

Deu certo quando `tabela` tem dez linhas e três colunas novas, cada uma com um valor que a classe aceitou.

Dez linhas vazias

```
ERRO DE AUTENTICAÇÃO - Chave de API inválida
...
  requerido polo_passivo resultado
0      None          None      None
1      None          None      None
2      None          None      None
```

O programa não para: cada linha que falha volta vazia, e o aviso fica acima da tabela. Quando é a chave, conferir o `.env` (o nome é `GROQ_API_KEY`, sem espaço em volta do =) e o passo 1.

Por trás do dataframeit

O que o dataframeit fez com as dez?

Uma chamada, dez linhas preenchidas. Com o que vocês já sabem de Python, como escreveriam isso sem a biblioteca?

Um `for`, uma chamada por linha

RESPOSTA

```
respostas = []
for texto in dez["texto"]:
    pedido = prompt.replace("{texto}", texto)
    resposta = modelo(pedido)      # vai à Groq e volta
    respostas.append(Formulario(resposta))  # confere
# e põe cada campo numa coluna da tabela
```

É um esquema, não o código da biblioteca: `modelo` aqui é a chamada à Groq.

O que vem além do `for`

O que a biblioteca faz	Onde aparece
conta os tokens de cada linha	`_input_tokens`, `_output_tokens`, `_reasoning_tokens`
anota a linha que falhou e por quê	`_dataframeit_status`, `_error_details`
tenta de novo quando o serviço recusa	o argumento `max_retries`, 3 por padrão
retoma do ponto em que parou	o argumento `resume`

Pensar mais, pagar mais

Tokens que não aparecem na resposta

Na primeira sentença, o modelo gastou 215 tokens de saída, e 153 deles são `_reasoning_tokens`. A resposta tem só três campos curtos.

O que o modelo escreveu nesses 153?

Um rascunho antes da resposta

RESPOSTA

Antes de preencher o formulário, o modelo escreve um raciocínio que não volta para nós: lê o caso, compara opções, confere. O esforço (`low`, `medium`, `high`) diz quanto ele pode escrever ali.

Como uma pessoa: pensar mais costuma dar resposta melhor, mas demora mais. E o rascunho conta como token de saída: se paga.

Três esforços, três sentenças

Esforço	Raciocínio	Tempo	Erro no requerido
`low`	0, 0, 0	13 s	na 1ª, o pedido no lugar do réu
`medium`	0, 0, 0	10 s	na 3ª, o pedido no lugar do réu
`high`	85, 31, 37	13 s	na 3ª, o mesmo erro

O mesmo formulário com o Luna, pelo Codex. Pensar mais não consertou a 3ª: o campo `requerido` não diz o que é um réu.

Quanto custaram as dez

gpt-oss-120b na Groq	Tokens	US$ por milhão
entrada	34.057	0,15
saída, com o raciocínio	2.495	0,60

As dez custariam menos de um centavo de dólar (US$ 0,0066); as 300, uns 20 centavos. Na conta gratuita, a Groq não cobra: limita quantos tokens passam por minuto, e por isso as dez levaram 3 minutos e meio.

Sem pagar token: Claude Code e Codex

Quem assina o Claude ou o ChatGPT pode usar a cota da assinatura no lugar da chave:

Assinatura	No dataframeit

Claude	`uv add "dataframeit[claude-code]"`
	`provider="claude_code"`
	`model="haiku"`

ChatGPT	`uv add "dataframeit[codex]"`
	`provider="codex"`
	`model="gpt-6-luna"`

Os dois caminhos lado a lado

Insper

Guardar antes de comparar: `to_csv`

```
dez = decisoes.head(10)
dez["requerido_regex"] = dez["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
dez.to_csv("dez-decisoes-regex.csv", index=False)
```

`to_csv` grava a tabela num arquivo; `index=False` deixa de fora a numeração das linhas. O caderno gravou `tabela`, a do modelo, do mesmo jeito. Com os dois arquivos salvos, a comparação não depende de rodar tudo de novo.

As mesmas dez, regex contra modelo

```
import pandas as pd

regex = pd.read_csv("dez-decisoes-regex.csv")
modelo = pd.read_csv("dez-decisoes-modelo.csv")

regex["requerido_modelo"] = modelo["requerido"]
print(regex[["requerido_regex", "requerido_modelo"]])
```

`read_csv` relê os dois arquivos. Onde as duas colunas diferem, quem acertou?

Quando usar cada um?

Olhando as dez sentenças: quando vocês usariam a regex, e quando o modelo?

Regra para o que tem forma, modelo para o que tem sentido

RESPOSTA

	Expressão regular	Modelo de linguagem
acerta	em texto de forma fixa	em texto variado
custa	testar a regra	uma chamada por linha
erra	em silêncio	com confiança
se confere	lendo o vazio	lendo uma amostra

Explicar cada campo

Onde o modelo errou?

No cabeçalho	O que o modelo escreveu em `requerido`
Fazenda Pública do Estado de São Paulo e outro	Fazenda Pública do Estado de São Paulo e Município de Marília
MUNICIPIO DE AMERICANA e outro	MUNICIPIO DE AMERICANA
Fazenda Pública do Estado de São Paulo e outro	Fazenda Pública do Estado de São Paulo

O que o formulário deixou de dizer ao modelo?

O campo não diz o que é um réu

RESPOSTA

`requerido: str` aceita qualquer texto. O modelo decidiu sozinho, a cada sentença, se listava os dois réus ou só um, e em que caixa escrevia. Nas sentenças de Americana e de Taquaritinga, o segundo réu está no dispositivo.

O que o modelo precisa saber vai escrito no próprio campo.

Segunda versão: `description` no requerido

```
class Formulario(BaseModel):
    requerido: str = Field(description=(
        "Todos os réus, separados por ' e '. "
        "Quando o cabeçalho diz 'e outro', "
        "procure no texto quem é o outro réu. "
        "Escreva cada nome como na sentença, "
        "com só a primeira letra de cada "
        "palavra maiúscula."))
```

... e no polo passivo

```
    polo_passivo: Literal["público", "privado"] = (
        Field(description=(
        "público se algum réu é ente público: "
        "União, estado, município, Fazenda "
        "ou autarquia; privado se todos são "
        "planos de saúde, operadoras ou "
        "empresas.")))
```

O `resultado` fica como estava: o modelo acertou as dez.

A segunda versão nas mesmas dez

Sentença	O que o modelo escreveu em `requerido`
Americana	Municipio De Americana e Fazenda Publica Do Estado De Sao Paulo
Taquaritinga	Fazenda Pública do Estado de São Paulo e Município de Taquaritinga
Jundiaí	Prefeitura Municipal De Jundiai e Estado De Sao Paulo

O segundo réu apareceu. A regra de maiúsculas virou "Do" e "De", e o acento sumiu em três. Explicar o campo conserta o que é leitura; grafia se padroniza depois, em Python.

Regex ou modelo?

Regex ou modelo? Itens 1 a 5

#	Em cada sentença, ...
1	... extrair o número do processo no padrão CNJ
2	... dizer se o juiz reconheceu dano moral
3	... pegar a data que vem no cabeçalho
4	... dizer se o atraso do voo foi culpa da companhia ou do tempo
5	... achar o valor da causa, escrito como "R$ 10.000,00"

Itens 1 a 5

RESPOSTA

#		Por quê
1	R	o número tem formato fixo
2	M	o juiz diz isso de muitos jeitos, inclusive negando
3	R	o cabeçalho tem lugar e formato fixos
4	M	o motivo está no meio da fundamentação
5	R	o valor tem forma fixa: R$, dígitos, vírgula

Regex ou modelo? Itens 6 a 10

#	Em cada sentença, ...
6	... dizer se o juiz achou a prova do autor suficiente
7	... contar quantas vezes aparece "art. 256"
8	... resumir em uma frase o argumento da defesa
9	... pegar o nome do juiz na linha "Juiz(a) de Direito:"
10	... dizer se a companhia ofereceu outro voo ao passageiro

Itens 6 a 10

RESPOSTA

#		Por quê
6	M	é um juízo, escrito de muitos jeitos
7	R	é um texto exato
8	M	resumir é escrever um texto novo
9	R	o rótulo é fixo, e o nome vem depois dele
10	M	a reacomodação aparece no relato, sem fórmula

Um cliente novo

Desta vez, a base não vem pronta

	Canabidiol	O cliente novo
a base	300 sentenças, prontas no CSV	coletamos nós
a busca	"canabidiol", escolhida antes	termos, tribunal e período: escolha nossa
a análise	três perguntas do cliente	as perguntas também são nossas

Vamos fazer algo parecido, do começo: o que perguntar, o que buscar e como analisar depois.

Uma grande companhia aérea

O escritório quer conquistar uma grande companhia aérea como cliente. A diretoria jurídica dela mandou uma mensagem curta:

> **Descobri que tem esse recurso no STF. Me conta como isso vai me afetar.**

O recurso é o ARE 1.560.244, o Tema 1.417. Está sem julgamento e sem data de pauta, e enquanto isso há processos suspensos no país inteiro.

O que a Constituição diz do transporte aéreo

Constituição Federal. Art. 178.

A lei disporá sobre a ordenação dos transportes aéreo, aquático e terrestre, devendo, quanto à ordenação do transporte internacional, observar os acordos firmados pela União, atendido o princípio da reciprocidade.

A pergunta do Tema 1.417, em resumo

Quando o voo atrasa, é cancelado ou muda por caso fortuito ou força maior, a responsabilidade da companhia segue o Código Brasileiro de Aeronáutica ou o Código de Defesa do Consumidor?

O STF decide à luz do art. 178. As companhias defendem o CBA, que lista no art. 256, § 3º, as situações que afastam a responsabilidade delas.

A suspensão nacional

Quando	O que o relator decidiu
26/11/2025	suspendeu, no país inteiro, os processos sobre a questão do tema, até o julgamento
10/03/2026	esclareceu o alcance: ficam parados os casos de meteorologia, infraestrutura do aeroporto, ordem de autoridade e pandemia (art. 256, § 3º, do CBA); falha da própria companhia, como manutenção ou overbooking, segue tramitando

Para a companhia, a mesma ação pode estar parada ou andando, conforme o motivo do atraso.

O que vale a pena perguntar

Insper

Três perguntas para a companhia

Vocês estão no escritório que quer conquistar a companhia. Escrevam três perguntas que vocês responderiam com dados para ela, e mandem pelo Blackboard.

Para cada pergunta: que dado vocês coletariam para respondê-la?

Qual de cada par vocês levariam à companhia, e por quê?

#	A	B
1	Coletar todas as ações contra a companhia e ver o que aparece	Quantas ações dela estão paradas pela suspensão?
2	Que palavras mais aparecem nas sentenças?	Em quantas o juiz culpa uma falha da própria companhia?
3	Cruzar os dados da ANAC com todas as sentenças	Quanto ela deixa de pagar se o STF escolher o CBA?

Pergunta de pesquisa e mineração de dados

RESPOSTA

	Mineração (A)	Pesquisa (B)
começa por	os dados que houver	a decisão do cliente
o que coletar	tudo	o que a pergunta pede
a resposta	um padrão, talvez inútil	um número conferível

Pesquisa ou mineração? Itens 1 a 5

#	Para a companhia aérea
1	Quantas ações contra ela no TJSP em 2025 tratam de cancelamento por mau tempo?
2	Baixar todas as sentenças com a palavra "voo" e ver o que aparece
3	Qual o valor médio da indenização por dano moral em atraso de voo nos Juizados de São Paulo?
4	Fazer uma nuvem de palavras das sentenças
5	Em quantas ações ela alega força maior, e em quantas o juiz aceita?

Itens 1 a 5

RESPOSTA

#		Por quê
1	P	diz o tribunal, o ano e o motivo: mede quantas ações o tema alcança
2	M	nem recorte nem número a responder
3	P	um número que entra na provisão da companhia
4	M	mostra palavras frequentes, que não mudam decisão nenhuma
5	P	diz se a defesa de força maior funciona

Pesquisa ou mineração? Itens 6 a 10

#	Para a companhia aérea
6	Juntar todas as bases da ANAC e procurar correlações
7	Quanto ela teria pago a menos em 2025 se valesse o CBA?
8	Rodar um modelo sobre as sentenças para achar padrões escondidos
9	Os juízes que aceitam o mau tempo como excludente aceitam também a falta de tripulação?
10	Listar os 100 termos mais frequentes nas petições

Itens 6 a 10

RESPOSTA

#		Por quê
6	M	começa pelos dados, sem saber o que procura
7	P	é o que está em jogo no STF, em reais
8	M	"padrão escondido" não diz qual pergunta responde
9	P	separa fortuito externo de interno, a linha da suspensão
10	M	frequência de termos sem pergunta por trás

As perguntas de vocês

Vamos ler as respostas do Blackboard e classificar juntos: pergunta de pesquisa ou mineração?

Quando for mineração, o que falta para virar pergunta: o recorte, o número que se quer, ou a decisão que ele muda?

O caminho até a resposta

Etapa	No caso da companhia
definir a amostra	que termo, tribunal e período
o que coletar	que dado de passageiro fica de fora
coletar	raspar e guardar, com a ficha da coleta
extrair e conferir	o motivo do atraso, e quantas o modelo acerta
limpar e juntar	um nome por companhia; sentença com o processo
analisar	responder à companhia nos dois desfechos do STF

As perguntas que vamos responder

Das perguntas de pesquisa que sobraram, quais o escritório leva à companhia?

Escolhemos juntos as que vamos responder até o fim do curso.

Na próxima aula

Como saber quantas ações o Tema 1.417 alcança sem ler todas: o que uma amostra diz sobre o todo, e o que uma consulta mal feita esconde.